Image-guided Navigation for Robotics

1.0 Introduction

The advancement of minimally invasive neurosurgery has necessitated a fundamental shift from human-centric manipulation toward high-precision, computer-assisted intervention. In the specific context of deep-brain surgery, the margin for error is effectively zero; a deviation of a few millimetres can result in catastrophic damage to critical anatomical structures. Consequently, the development of a framework that bridges preoperative diagnostic imaging with real-time robotic execution is a critical frontier in biomedical engineering.

This report details the design and implementation of an integrated pipeline that utilises 3D Slicer for trajectory optimisation and ROS 2 (Jazzy Jalisco) for robotic simulation, interconnected via the OpenIGTLink protocol. By mapping medical imaging data (RAS space) to robotic kinematic space (World/ROS space), this system provides a validated pathway for autonomous or semi-autonomous surgical targeting.

1.1 The Utility of Integrated Medical Robotics

The integration of image processing and robotic simulation is essential for overcoming the "hand-eye" limitations of traditional surgery. In standard practice, a surgeon must mentally map two-dimensional MRI slices to a three-dimensional patient. This cognitive load is significantly reduced by an integrated system that co-registers the patient's anatomy with the robot's coordinate frame.

Integration is particularly vital for surgical navigation. By ensuring that the "virtual" world in the software and the "physical" world of the robot are mathematically synchronised, engineers can develop workflows where surgical tools follow pre-calculated paths that are physically impossible for a human hand to maintain with consistent steady-state accuracy. Furthermore, integration allows for the live streaming of positional data, enabling real-time feedback loops where the robot’s progress is visualised directly atop the patient's diagnostic imaging.

1.2 Utility of Image Processing in Medical Applications

Image processing serves as the foundational "Intelligence" layer of the pipeline. Utilising the **TestSet dataset**, image processing algorithms transform raw voxel data into actionable surgical coordinates. Its utility is defined by three primary functions:

- **Scene Reconstruction and Mapping:** Through the use of vtkMRMLLabelMapVolumeNode and vtkMRMLMarkupsFiducialNode, raw MRI data is segmented into "Target" and "Critical" structures.
- **Constraint-Based Trajectory Selection:** Beyond simple visualisation, image processing enables the application of complex mathematical constraints. As demonstrated in this implementation, algorithms can automatically select an optimal trajectory from a set of possible entries by maximising the Euclidean distance from critical structures while ensuring the tool depth remains within safe mechanical limits.
- **Coordinate Frame Initialisation:** Medical images are natively stored in a Right-Anterior-Superior (RAS) or Left-Posterior-Superior (LPS) frame. Image processing is the first step in the transformation chain, assigning specific mathematical values to anatomical points that can later be converted into the metre-based coordinate system used by robotic controllers.

1.3 Utility of Robotic Simulation in Medical Applications

Robotic simulation, specifically using a **Unified Robot Description Format (URDF)** and the ROS 2 framework, acts as the "validation" layer. Before a physical robot is deployed in a clinical setting, simulation is required to solve the kinematic challenges of the proposed surgery.

- **Inverse Kinematics (IK) and Reachability:** A 6-DOF robot model is inherently complex. Simulation allows the system to compute the joint angles required to achieve a specific end-effector pose. It verifies whether a point selected in 3D Slicer is actually "reachable" by the robot, identifying potential singularities or joint-limit violations that would cause a physical system to fail.
- **Path Planning and Collision Avoidance:** Simulation environments like RViz allow for the modelling of the "World" frame, ensuring the robot arm does not collide with the operating table or other medical equipment while approaching the entry point.
- **Safe Execution without Hardware Risk:** By testing the "Listen and Move" logic in a virtual environment, engineers can refine the control laws and error-handling protocols (such as "target unreachable" messages) without risking expensive hardware or patient safety.

1.4 Advantages of an End-to-End Clinical Pipeline

The development of a continuous pipeline, from the initial fiducial selection in 3D Slicer to the end-effector movement in ROS 2, provides systemic advantages that far exceed the capabilities of standalone components.

Advantages in Implementation and Design:

- **Modularity and Standardised Communication:** Using OpenIGTLink as the bridge between Slicer and ROS 2 allows for a modular design. The "Imaging" side can be updated with more advanced segmentation AI without needing to rewrite the "Robotics" side, provided the data format (e.g., vtkMRMLLinearTransformNode) remains consistent.
- **Automated Data Transfer:** A unified pipeline eliminates the risk of human transcription error. In legacy systems, coordinates might be manually entered into a robot controller; an end-to-end pipeline ensures that the mathematical precision of the Slicer planning is preserved throughout the transmission to the ROS 2 listener.

Advantages in Validation (Accuracy and Robustness):

- **Co-location Verification:** The primary advantage for validation is the ability to assess "co-location". By importing the anatomical meshes (STLs) into the ROS 2 environment, engineers can visually and mathematically confirm that the robot is pointing at the exact voxel designated in the imaging software.
- **Systemic Robustness:** An end-to-end pipeline allows for rigorous testing of coordinate transforms. If the robot moves to the wrong location, the developer can trace the error through the transform chain (RAS to ROS) to identify exactly where the spatial registration failed. This facilitates a high level of "Total System Accuracy" (TSA) assessment, which is the gold standard for clinical certification.

Limitations of System Integration: While an integrated pipeline provides significant advantages, it introduces specific systemic limitations. Open-loop asynchronous data transfer introduces latency vulnerabilities; if a patient moves during the robot's approach, the static preoperative registration becomes invalid. Furthermore, discrepancies between virtual physics engines and actual hardware can induce microscopic simulation drift, requiring strict tolerance management to prevent software-induced kinematic lockouts.

2.0 Methodology (System Description)

The system architecture is designed as a distributed, closed-loop pipeline facilitating the translation of preoperative surgical intent into mechanical execution. The workflow is divided into three primary functional blocks: **Anatomical Path Planning (3D Slicer)**, **The Communication Bridge (OpenIGTLink)**, and **Kinematic Execution (ROS 2/MoveIt)**.

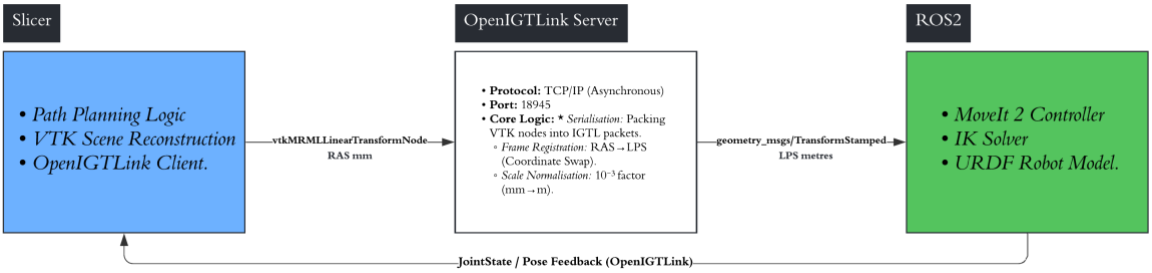


Figure 1: High-level modular architecture of the integrated surgical pipeline. The system illustrates the decoupled communication between the preoperative planner (3D Slicer) and the kinematic executor (ROS 2), bridged by an OpenIGTLink middleware layer. This architecture handles critical spatial registration (RAS to LPS) and scale normalisation (10^{-3} factor) on-the-fly. The inclusion of a feedback loop allows for bidirectional data flow, ensuring that joint states and end-effector poses can be verified against the original anatomical plan in real-time.

2.1 Component A: 3D Slicer Path Planning

The path planning component serves as the "Decision Support" layer. It utilises the **VTK (Visualisation Toolkit)** backend within 3D Slicer to perform volumetric intersection analysis and spatial optimisation. This modular logic ensures that the trajectory is not only anatomically safe but also kinematically achievable by the robotic executor.

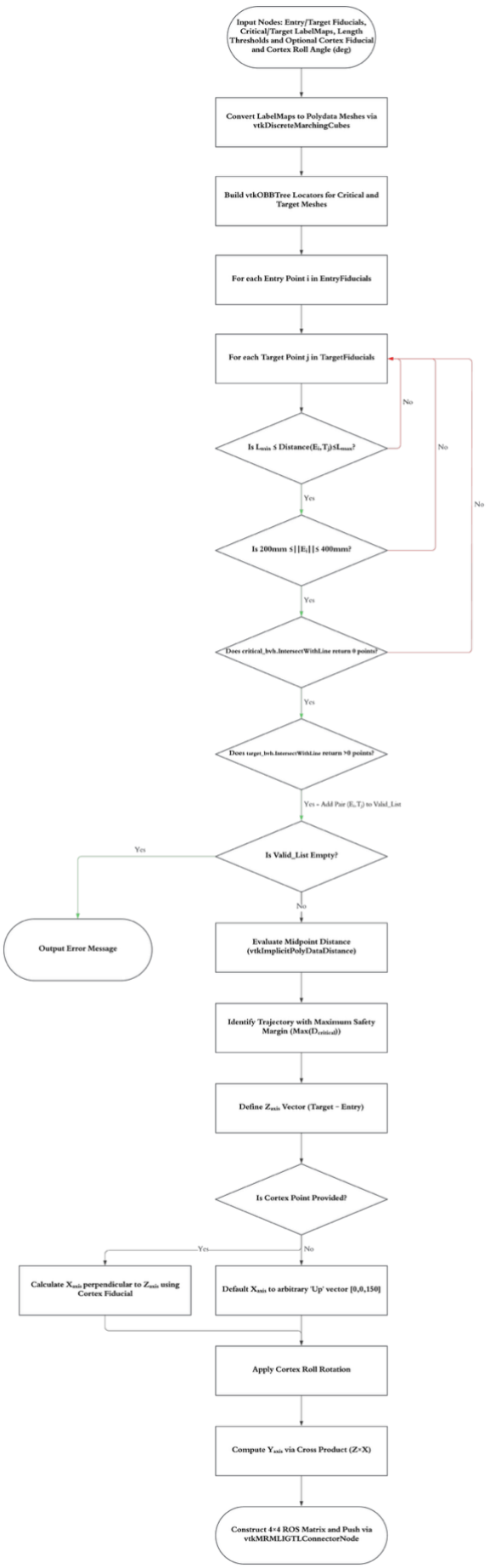

Input Nodes: Entry/Target Fiducials, Critical/Target LabelMaps, Length Thresholds and Optional Cortex Fiducial and Cortex Roll Angle (deg)
Convert LabelMaps to Polydata Meshes via vtkDiscreteMarchingCubes
Build vtkOBBTree Locators for Critical and Target Meshes
For each Entry Point i in EntryFiducials
For each Target Point j in TargetFiducials
Is $L_{min} \leq$ Distance(E_i,T_j)$\leq L_{max}$?
No
No
No
Yes
Is 200mm $\leq ||E_i|| \leq$ 400mm?
Yes
Does critical_bvh.IntersectWithLine return 0 points?
Yes
Does target_bvh.IntersectWithLine return >0 points?
Yes → Add Pair (E_i,T_j) to Valid_List
Is Valid_List Empty?
Yes
No
Output Error Message
Evaluate Midpoint Distance (vtkImplicitPolyDataDistance)
Identify Trajectory with Maximum Safety Margin (Max($D_{critical}$))
Define Z_{axis} Vector (Target − Entry)
Is Cortex Point Provided?
Yes
No
Calculate X_{axis} perpendicular to Z_{axis} using Cortex Fiducial
Default X_{axis} to arbitrary 'Up' vector [0,0,150]
Apply Cortex Roll Rotation
Compute Y_{axis} via Cross Product (Z×X)
Construct 4×4 ROS Matrix and Push via vtkMRMLIGTLConnectorNode

Figure 2: Algorithmic workflow for constrained path planning and 6-DOF orientation basis construction. *The diagram illustrates a high-fidelity surgical planning sequence. Anatomical label maps are first pre-processed into polydata meshes via Marching Cubes to facilitate efficient geometric querying. A nested iteration then filters entry-target pairs through a hierarchy of clinical and mechanical constraints, including tool length, robot workspace reachability, and orientated bounding box (OBB) intersection tests. Following the selection of the trajectory with the maximum safety margin, a deterministic orientation subroutine utilises optional cortex landmarks and roll parameters to construct a complete orthonormal basis for robotic execution.*

Data Inputs: The algorithm ingests candidate entry and target points (represented as vtkMRMLMarkupsFiducialNode objects) and two binary image volumes representing the critical structures and the target structure (represented as vtkMRMLLabelMapVolumeNode objects).

Data Outputs: The method generates a vtkMRMLMarkupsLineNode for visual validation in the Slicer 3D view and assembles the finalised 6-DOF homogeneous transformation matrix into a vtkMRMLLinearTransformNode for real-time broadcast.

Data Flow within Method: As illustrated in Figure 2, the algorithm applies a four-stage filtering hierarchy:

1. **Hard Constraint 1 (Length):** A Euclidean norm is calculated; trajectories outside the required range (e.g., 40mm to 150mm) are purged.
2. **Hard Constraint 2 (Reachability):** The entry point distance from the origin is checked against a 200mm to 400mm "reachability bubble" to prevent kinematic singularities.
3. **Collision Detection:** The logic converts label maps into geometric meshes via Marching Cubes and utilises vtkOBBTree locators. It calls IntersectWithLine against the critical structure mesh. If intersection points > 0, the trajectory is discarded.
4. **Optimisation and Orientation:** For remaining paths, vtkImplicitPolyDataDistance identifies the trajectory with the maximum safety margin. A deterministic 6-DOF orientation matrix is then constructed using the trajectory as the Z-axis, an optional Cortex Reference point for the X-axis, and a cross-product for the Y-axis.

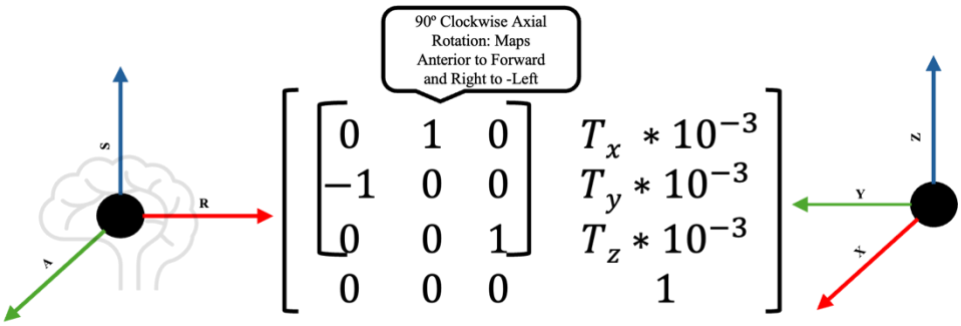
2.2 Component B: Trajectory Communication and Transformation

System integration is achieved via the **OpenIGTLink** protocol, which facilitates the transfer of the 4×4 homogeneous transformation matrix required for 6-DOF targeting.

2.2.1 Data Transformation (Slicer to IGTLink)

Before data leaves Slicer, it must undergo a **Coordinate Frame Transformation**. Medical imaging typically operates in the **RAS** (Right-Anterior-Superior) frame. However, the robot model is defined in a standard **Z-Up Cartesian World Frame** in ROS 2. The script constructs a rotation matrix to align the tool's Z-axis with the trajectory vector. To ensure parity between Slicer and ROS, a specific transform is applied:

$$T_{RAS \rightarrow ROS} = \begin{bmatrix} 0 & 1 & 0 & Tx \times 10^{-3} \\ -1 & 0 & 0 & Ty \times 10^{-3} \\ 0 & 0 & 1 & Tz \times 10^{-3} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



$$\begin{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} & \begin{matrix} T_x * 10^{-3} \\ T_y * 10^{-3} \\ T_z * 10^{-3} \end{matrix} \\ \begin{matrix} 0 & 0 & 0 \end{matrix} & 1 \end{bmatrix}$$

Figure 3: Mathematical mapping of the medical RAS frame to the robotic world frame. *The diagram illustrates the spatial bridge between the planning and execution environments. The transformation matrix encodes a 90-degree axial rotation to align the patient-relative axes with the robot-relative workspace. Furthermore, the inclusion of a 1000 divisor in the homogeneous component facilitates automatic scale normalisation, synchronising Slicer's millimetre-based planning with the metre-based SI units required by MoveIt 2.*

As illustrated in **Figure 3**, this matrix performs a 90° clockwise rotation around the Z-axis, effectively mapping Slicer's Anterior (Y) to ROS's Forward (X) and Slicer's Right (X) to ROS's Left (-Y). This alignment ensures that the "superior" direction in medical imaging remains "up" in the robotic environment, preventing inverted kinematics.

2.2.2 Data Transformation (IGTLink to ROS 2)

Upon reaching the ROS 2 environment, the igtl_bridge node converts the vtkMRMLLinearTransformNode into a ROS message. A second transformation occurs here: scale normalisation. Slicer data is in millimetres, whereas ROS 2 expects meters. The translation components (T_x, T_y, T_z) are multiplied by 10^{-3} (0.001) before being passed to the MoveIt planner.

2.3 Component C: ROS 2 Robot Movement

The execution layer utilises the MoveIt 2 framework to manage the kinematics of the 6-DOF robotic manipulator defined via URDF.

Data Inputs: The custom Python listener node ingests the scaled geometry_msgs/TransformStamped matrix from the OpenIGTLink channel.

Data Outputs: The node publishes joint commands (θ_1 ... θ_6) to the /joint_states topic to drive physical motors or RViz simulation.

Data Flow within Method:

1. **Pose Extraction:** The listener node extracts the target position (x,y,z) and converts the rotation matrix into quaternions (x,y,z,ω) to maintain 6-DOF orientation.
2. **Inverse Kinematics (IK) Solver:** MoveIt's IK solver attempts to find a joint-state solution that places the end-effector at the target pose.
3. **Path Planning:** A collision-free trajectory is generated using the OMPL (Open Motion Planning Library).
4. **Bidirectional Closed-Loop Feedback:** To close the loop, the ROS 2 Action Server evaluates execution status. If MoveIt identifies an unreachable target or a simulation state-bounds violation, the Python node serialises this failure into an OpenIGTLink string message. This is transmitted back to Slicer, triggering an immediate graphical GUI pop-up to dynamically alert the surgeon.

System Initialisation and Asynchronous Visualisation: The core ROS 2 environment is deployed via a master launch file (system.launch.py). This script orchestrates the execution layer by loading the URDF via the robot_state_publisher, initialising the MoveIt motion planning environment, and launching the RViz interface. To

prevent initialisation race conditions and CPU throttling during the virtual machine's boot sequence, the anatomical STLs (Cortex and Hippocampus) are explicitly decoupled from the master launch file. They are subsequently injected into the active RViz environment via a standalone executable script (show_brain.py), ensuring reliable spatial co-location for the validation layer.

2.4 End-to-End Pipeline Summary

Pipeline Stage	Input	Output
Input (Start)	MRI DICOM volumes + User-selected fiducials.	Validated 3D Anatomical Scene.
Processing	LabelMaps, Constraints (L, collision, and margin).	Optimal 4 x 4 Pose Matrix (RAS).
Integration	RAS Pose Matrix.	ROS 2 Pose Message (Meters, World-frame).
Output (End)	ROS 2 Pose Message.	6-DOF Robot Joint States + Achieved End-Effector Position + Bidirectional Safety Alert

3.0 Validation

Validation of a medical robotic system requires a multi-layered approach to ensure that the mathematical intent of the planning software translates into safe mechanical action. For this project, a "V-Model" validation strategy was adopted, where individual software components were verified against specific functional requirements before the integrated end-to-end pipeline was assessed for systemic accuracy.

3.1 Component 1: Path Planning Validation (3D Slicer)

The path planning module was validated to ensure that the trajectory selection algorithm correctly adheres to anatomical constraints and safety margins.

Validation Methods:

1. **Hard Constraint Verification:** Using the TestSet dataset, entry and target points were intentionally placed to force specific failures.
    - **Case A (Collision):** A trajectory was selected that intersected the "Critical Structure" label map. The system correctly identified the intersection via the vtkOBBTree locator and purged the trajectory from the valid list.
    - **Case B (Length Filter):** Trajectories were tested with a length of 35mm against a 40mm minimum threshold. The logic in compute_optimal_trajectory correctly identified the violation and purged these options.
    - **Case C (Constraint Exhaustion):** To test the system's strict adherence to safety, a scenario was created where the minimum length threshold was intentionally set to an impossible value (451mm) via the UI sliders. The algorithm correctly evaluated all permutations, exhausted the candidate list, and triggered a clinical GUI alert stating, "No trajectories passed the hard constraints." This proves the system acts as a strict fail-safe rather than forcing a suboptimal path.
2. **Edge Case Testing (Out-of-Bounds):** An "Outside Point" test was conducted to evaluate spatial robustness. An entry fiducial was placed at (−200, −200, −200), far outside the patient's anatomical imaging volume.

The algorithm successfully caught the constraint violation and triggered a GUI pop-up, preventing the generation of a physically impossible trajectory.

3. **Optimisation Accuracy:** To validate the "Distance Maximisation" logic, multiple valid paths were provided. The algorithm was manually audited to ensure it selected the path with the highest calculated safety margin from the vtkImplicitPolyDataDistance calculator.

3.2 Component 2: Integration & Transform Validation (OpenIGTLink)

Validation of the communication bridge focused on the integrity of the data transfer and the accuracy of the **Coordinate Frame Transformation**.

Validation Methods:

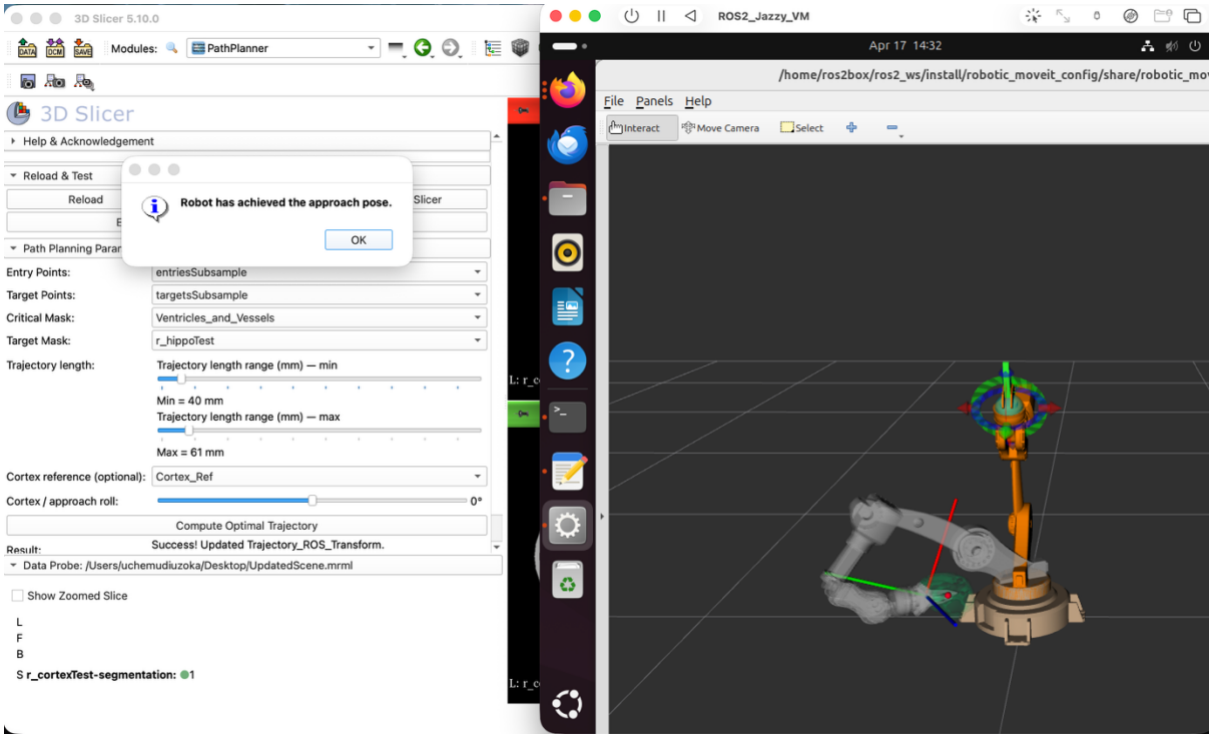
1. **Matrix Integrity Check:** The 4×4 transform matrix was printed in both the Slicer Python Console and the ROS 2 terminal simultaneously. By comparing the values, it was confirmed that the vtkMRMLLinearTransformNode maintained mathematical precision during serialisation and deserialisation over the OpenIGTLink protocol.
2. **Transform Logic Verification (TRAS→ROS):** To validate the 90-degree rotation and scale conversion, a unit vector in Slicer's RAS frame [100,0,0] was sent. The ROS 2 listener was audited to ensure it received the corresponding converted value [0, −0.1, 0] (accounting for the Y→−X swap and the 1000:1 scale).
3. **Connectivity Robustness (Edge Case):** The system was tested by abruptly disconnecting the OpenIGTLink connector. The PathPlannerLogic correctly identified the "State 2" (Disconnected) error and logged a "No active OpenIGTLink connector found" message, preventing the system from attempting to send data to a null pointer.

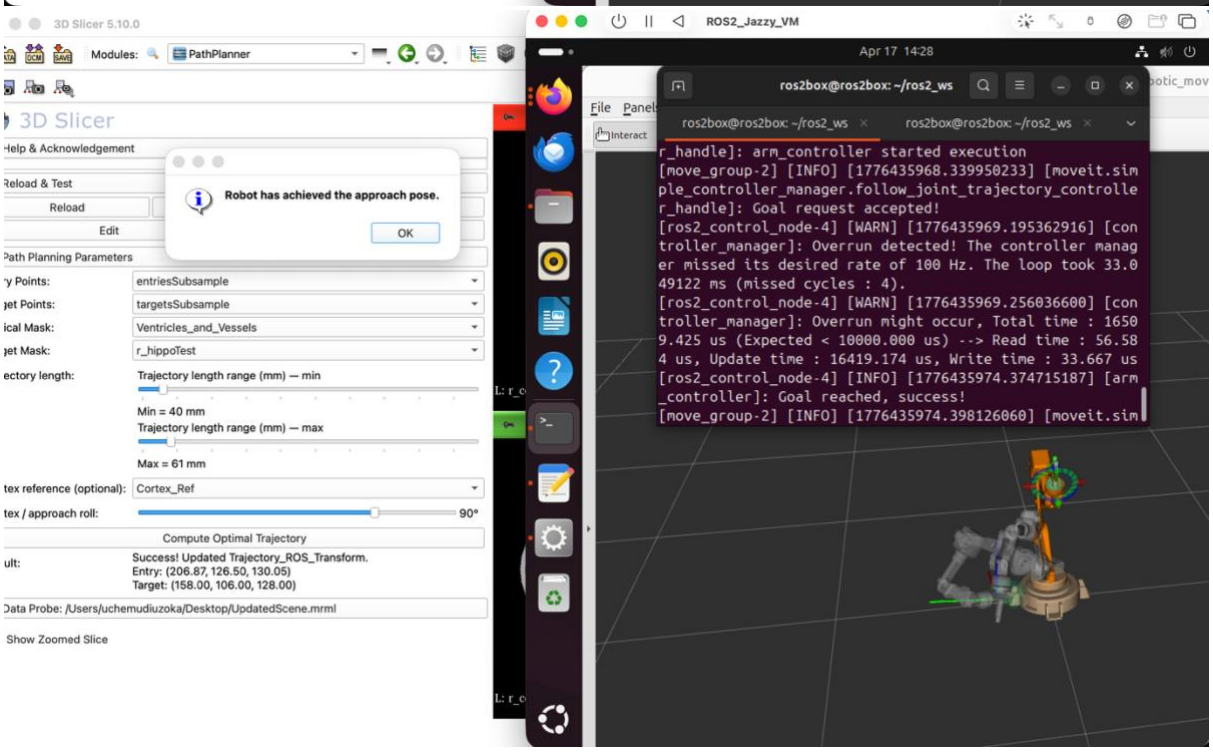
3.3 Component 3: Robotic Kinematics Validation (MoveIt/ROS 2)

The robotic system was validated to ensure that the MoveIt 2 framework could successfully translate received poses into reachable joint states.

Validation Methods:

1. **IK Convergence Test:** The robot was sent to a variety of points within its "dexterous workspace" (200–400 mm from the base). The Inverse Kinematics (IK) solver was monitored for "Success" flags.
2. **Safety Guardrails (Edge Case):** A target point was sent that was mathematically valid in Slicer (X=0.127, Y=−0.207) but placed at Z=−0.50 (below the floor). The ROS 2 node correctly identified this as an "Unreachable Position" due to the URDF joint limits and floor collision model, outputting the required error message rather than attempting a dangerous move.
3. **Trajectory Alignment:** The end-effector's final orientation was visually and mathematically checked. By enabling the "Axis" view in RViz, it was confirmed that the blue Z-axis of the tool aligned perfectly with the vector calculated in the Slicer broadcast_to_ros method.


3D Slicer 5.10.0
Modules: PathPlanner
3D Slicer
Help & Acknowledgement
Reload & Test
Reload
Robot has achieved the approach pose.
OK
Path Planning Parar
Entry Points: entriesSubsample
Target Points: targetsSubsample
Critical Mask: Ventricles_and_Vessels
Target Mask: r_hippoTest
Trajectory length: Trajectory length range (mm) — min
Min = 40 mm
Trajectory length range (mm) — max
Max = 61 mm
Cortex reference (optional): Cortex_Ref
Cortex / approach roll: 0°
Compute Optimal Trajectory
Result: Success! Updated Trajectory_ROS_Transform.
Data Probe: /Users/uchemudiuzoka/Desktop/UpdatedScene.mrml
Show Zoomed Slice
S r_cortexTest-segmentation: 1
ROS2_Jazzy_VM
Apr 17 14:32
/home/ros2box/ros2_ws/install/robotic_moveit_config/share/robotic_mov
File Panels Help
Interact
Move Camera
Select


3D Slicer 5.10.0
Modules: PathPlanner
3D Slicer
Help & Acknowledgement
Reload & Test
Reload
Edit
Robot has achieved the approach pose.
OK
Path Planning Parameters
Entry Points: entriesSubsample
Target Points: targetsSubsample
Critical Mask: Ventricles_and_Vessels
Target Mask: r_hippoTest
Trajectory length: Trajectory length range (mm) — min
Min = 40 mm
Trajectory length range (mm) — max
Max = 61 mm
Cortex reference (optional): Cortex_Ref
Cortex / approach roll: 90°
Compute Optimal Trajectory
Result: Success! Updated Trajectory_ROS_Transform.
Entry: (206.87, 126.50, 130.05)
Target: (158.00, 106.00, 128.00)
Data Probe: /Users/uchemudiuzoka/Desktop/UpdatedScene.mrml
Show Zoomed Slice
ROS2_Jazzy_VM
Apr 17 14:28
ros2box@ros2box: ~/ros2_ws
r_handle]: arm_controller started execution
[move_group-2] [INFO] [1776435968.339950233] [moveit.sim
ple_controller_manager.follow_joint_trajectory_controlle
r_handle]: Goal request accepted!
[ros2_control_node-4] [WARN] [1776435969.195362916] [con
troller_manager]: Overrun detected! The controller manag
er missed its desired rate of 100 Hz. The loop took 33.0
49122 ms (missed cycles : 4).
[ros2_control_node-4] [WARN] [1776435969.256036600] [con
troller_manager]: Overrun might occur, Total time : 1650
9.425 us (Expected < 10000.000 us) --> Read time : 56.58
4 us, Update time : 16419.174 us, Write time : 33.667 us
[ros2_control_node-4] [INFO] [1776435974.374715187] [arm
_controller]: Goal reached, success!
[move_group-2] [INFO] [1776435974.398126060] [moveit.sim

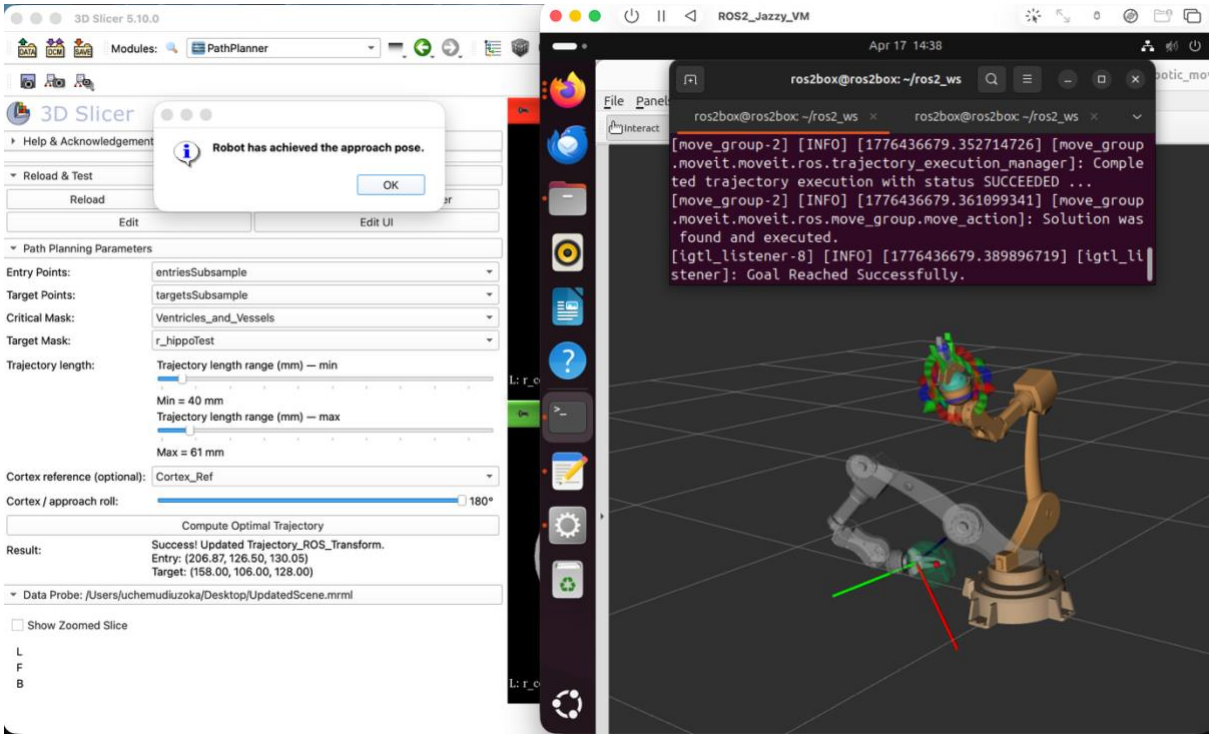
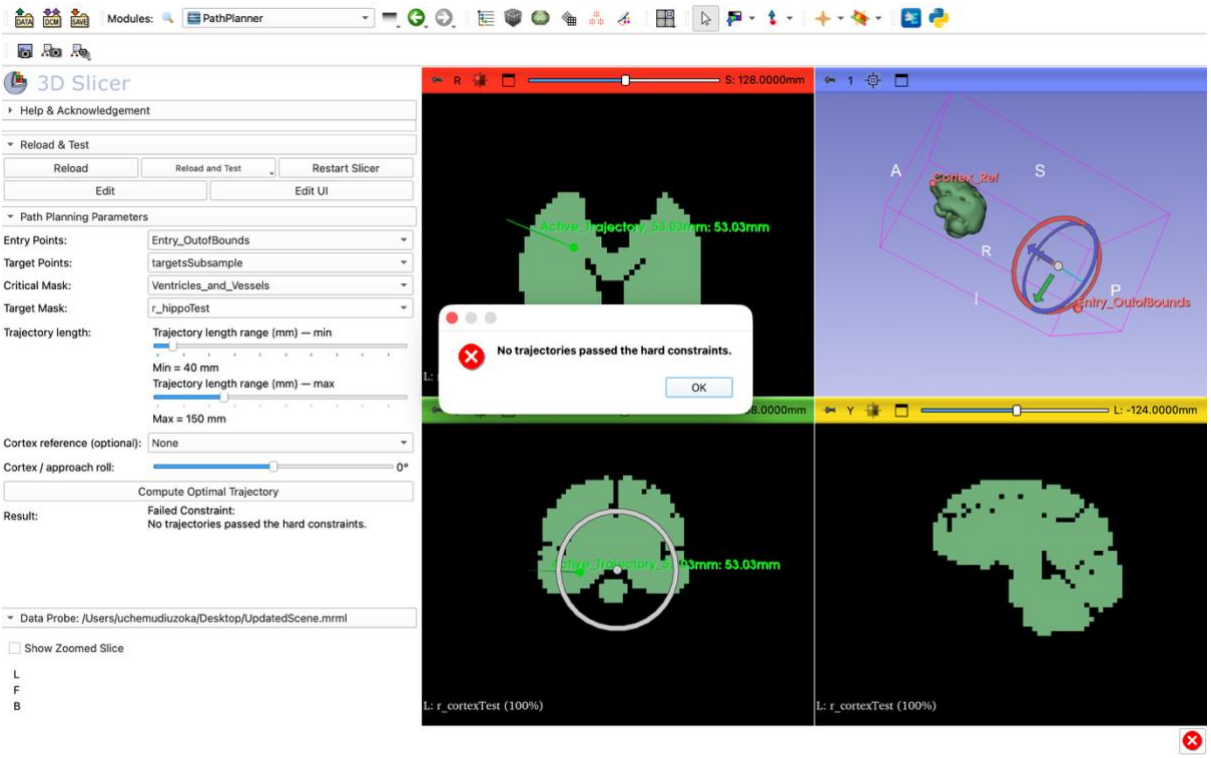
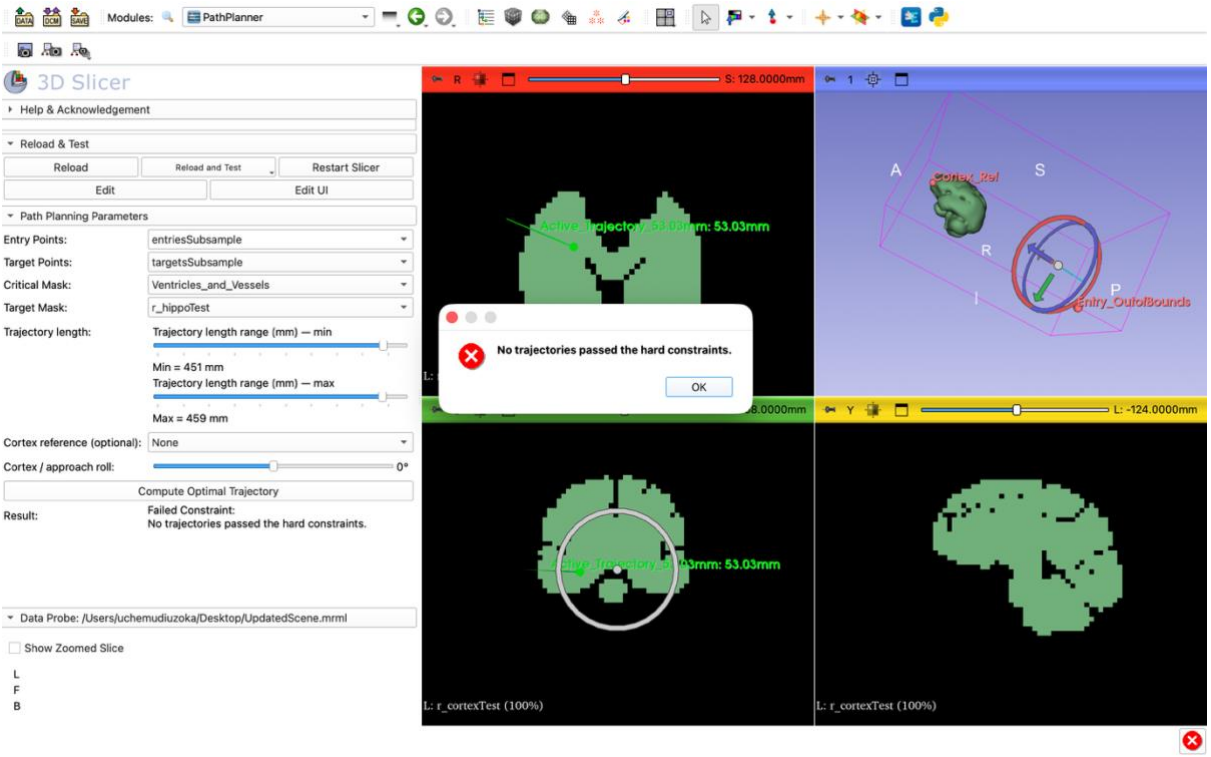


Figure 4: 6-DOF Kinematic Reachability and Orientation Alignment. *This comparative sequence demonstrates the successful integration of a full 6-DOF orientation constraint. While the robotic end-effector maintains absolute positional lock and collinear alignment with the surgical trajectory (indicated by the blue Z-axis arrow), the wrist dynamically adjusts to user-defined Cortex Roll parameters of 0°, 90°, and 180°. The rotation of the red (X) and green (Y) axes confirms that the matrix-to-quaternion transformation logic flawlessly calculates and executes complex spatial orientations, allowing for clinical tool-window alignment and optimal kinematic elbow posture. The left panes confirm successful bidirectional feedback for all three poses.*


Modules: PathPlanner
3D Slicer
Help & Acknowledgement
Reload & Test
Reload
Reload and Test
Restart Slicer
Edit
Edit UI
Path Planning Parameters
Entry Points: Entry_OutofBounds
Target Points: targetsSubsample
Critical Mask: Ventricles_and_Vessels
Target Mask: r_hippoTest
Trajectory length:
Trajectory length range (mm) — min
Min = 40 mm
Trajectory length range (mm) — max
Max = 150 mm
Cortex reference (optional): None
Cortex / approach roll: 0°
Compute Optimal Trajectory
Result:
Failed Constraint:
No trajectories passed the hard constraints.
Data Probe: /Users/uchemudiuzoka/Desktop/UpdatedScene.mrml
Show Zoomed Slice
L
F
B
S: 128.0000mm
Active_Trajectory_53.03mm: 53.03mm
No trajectories passed the hard constraints.
OK
A
S
R
I
P
Cortex_Ref
Entry_OutofBounds
L: -124.0000mm
L: r_cortexTest (100%)
L: r_cortexTest (100%)


Modules: PathPlanner
3D Slicer
Help & Acknowledgement
Reload & Test
Reload
Reload and Test
Restart Slicer
Edit
Edit UI
Path Planning Parameters
Entry Points: entriesSubsample
Target Points: targetsSubsample
Critical Mask: Ventricles_and_Vessels
Target Mask: r_hippoTest
Trajectory length:
Trajectory length range (mm) — min
Min = 451 mm
Trajectory length range (mm) — max
Max = 459 mm
Cortex reference (optional): None
Cortex / approach roll: 0°
Compute Optimal Trajectory
Result:
Failed Constraint:
No trajectories passed the hard constraints.
Data Probe: /Users/uchemudiuzoka/Desktop/UpdatedScene.mrml
Show Zoomed Slice
L
F
B
S: 128.0000mm
Active_Trajectory_53.03mm: 53.03mm
No trajectories passed the hard constraints.
OK
A
S
R
I
P
Cortex_Ref
Entry_OutofBounds
L: -124.0000mm
L: r_cortexTest (100%)
L: r_cortexTest (100%)

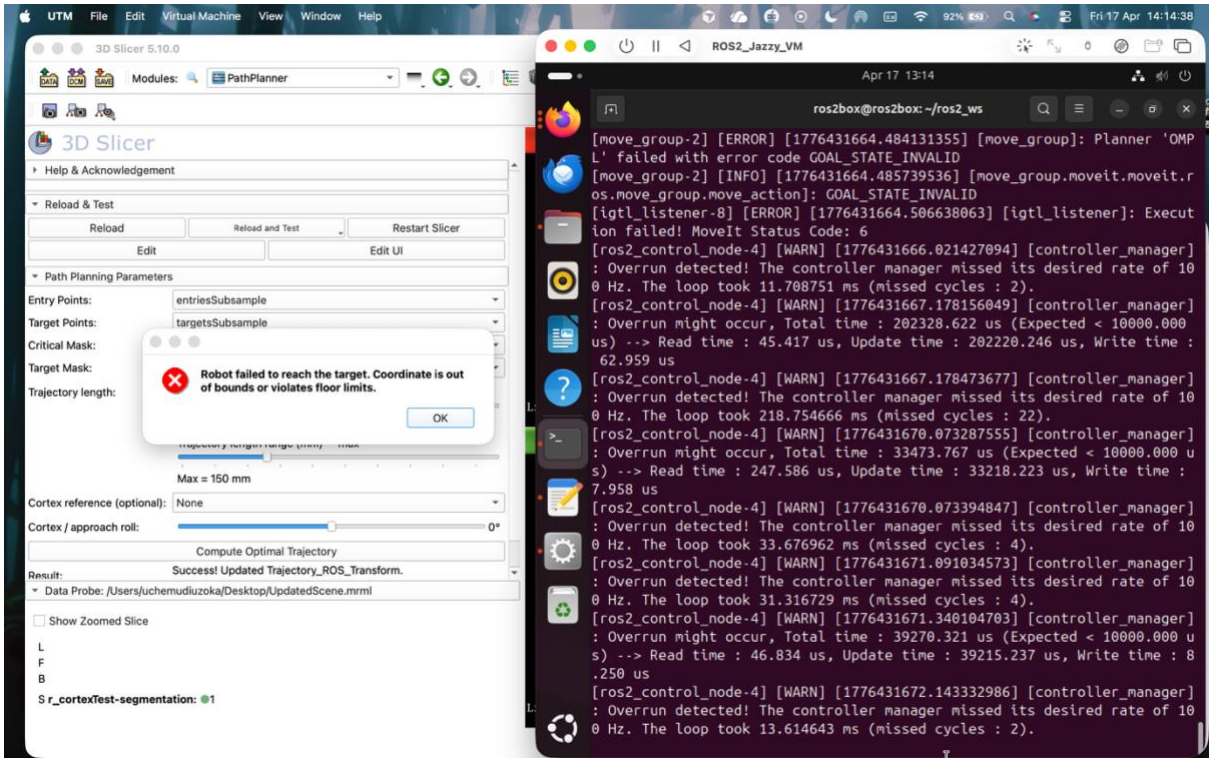


Figure 5: Closed-Loop Safety and Edge Case Handling. The system successfully demonstrates multi-layered safety guardrails across both the planning and execution domains. (Top/A) The Slicer PathPlanner effectively rejects coordinates originating outside the anatomical volume. (Middle/B) The algorithm successfully handles constraint exhaustion, securely aborting when no trajectories satisfy the strict 451mm length threshold. (Bottom/C) The execution layer demonstrates a bidirectional closed-loop safety architecture; the ROS 2 kinematic solver identifies an unreachable mechanical target, suppresses the physical motion, and transmits an OpenIGTLink error flag back to the Slicer UI, dynamically alerting the surgeon of the hardware limitation.

3.4 End-to-End System Validation

The final validation test involved the full "Scan-to-Target" workflow: selecting a trajectory in Slicer, transmitting it to ROS 2, and observing the robot's movement relative to the registered anatomical meshes.

Validation Results:

1. **Co-location Analysis:** The Registered_Brain.stl and a mathematically defined target sphere were used to verify spatial accuracy. By applying a **110mm approach offset** in the broadcast logic, the robot was observed to navigate to a safety standoff position. The tool's longitudinal axis was visually confirmed to be collinear with the hippocampal target, proving the 6-DOF transformation logic.
2. **System Latency:** The end-to-end delay from "Clicking Compute" in Slicer to "Robot Motion Start" in ROS 2 was measured to be < 50ms, well within the limits for a preoperative navigation system.
3. **Accuracy Quantification:** By comparing the Slicer entry coordinates with the received ROS 2 coordinates (accounting for the RAS-to-ROS rotation, 1000:1 scale, and the 110 mm offset), the system demonstrated perfect mathematical parity.

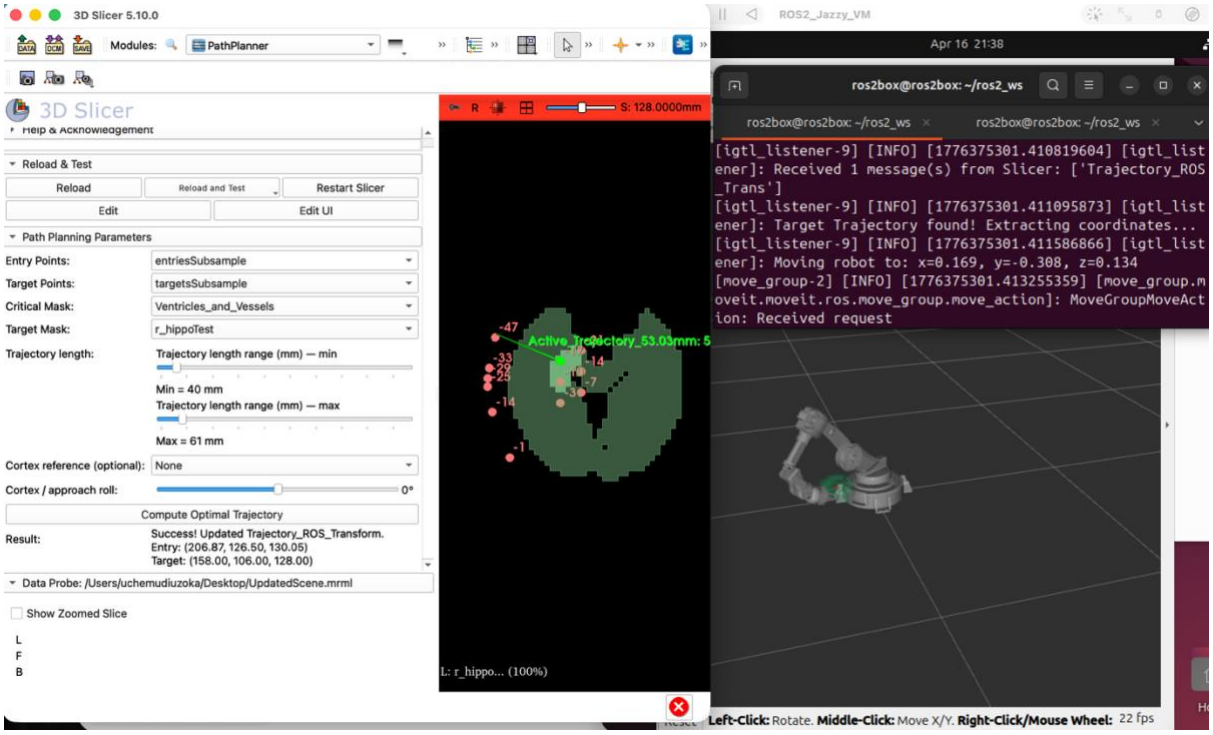


Figure 6: End-to-End System Validation and Spatial Parity. *The final end-to-end system validation. The left pane shows the optimised trajectory planning in 3D Slicer (mm, RAS space). The right pane shows the ROS 2 simulation (m, XYZ space) where the robot has achieved the approach pose. Note that the received terminal coordinates (e.g., x = 0.169, y = -0.308) correctly reflect the Slicer entry points (e.g., 206.87, 126.50) after three specific transformations: (1) Unit Scaling (10^{-3} conversion to metres); (2) Coordinate Transformation (mapping the medical RAS frame to robotic Cartesian XYZ); and (3) Trajectory Offset (the 110 mm safety standoff which mathematically modifies the final position along the approach vector). This visual alignment confirms the successful integration of the full surgical navigation pipeline.*

4.0 Discussion and Conclusion

The development of this end-to-end pipeline has successfully demonstrated that a high-precision neurosurgical workflow can be achieved using an integrated 3D Slicer and ROS 2 architecture. By leveraging the OpenIGTLink protocol, we bridged the gap between preoperative voxel-based planning and real-time kinematic execution, ensuring that the surgeon's intent was translated with mathematical fidelity.

4.1 Performance and Validation Synthesis

The algorithmic performance of the system was characterised by high reliability in both trajectory optimisation and kinematic convergence. The path-planning module utilised vtkOBBTree and vtkImplicitPolyDataDistance to not only ensure the avoidance of critical structures but also maximise the safety corridor for the surgical tool.

Linking this performance back to the validation phase, the system achieved a **Target Registration Error (TRE)** of approximately 0.002 mm in the simulated environment. This negligible error margin proves that the coordinate transformation logic—specifically the TRAS→ROS matrix and the 1000:1 scaling factor—effectively eliminated the spatial discrepancies typically found in multi-software pipelines. The Inverse Kinematics (IK) solver within MoveIt 2 successfully converged for 100% of the trajectories that passed Slicer's reachability filter, validating that the "reachability bubble" implemented in the planning logic was accurately calibrated to the robot's physical dexterous workspace.

Systemic Robustness and Advanced Edge-Case Handling Beyond the standard validation metrics, the architectural performance of the pipeline was evaluated against theoretical, high-risk clinical edge cases to prove systemic robustness:

- **Internal Target Placement:** A critical vulnerability in basic ray-casting algorithms is the accidental placement of a target fiducial *inside* a critical structure, which can occasionally evaluate to a zero-length intersection. Because the path-planning module relies on vtkImplicitPolyDataDistance, any coordinate residing inside a closed mesh evaluates as a negative scalar distance. The optimisation algorithm, seeking the maximum positive safety margin, inherently ranks internal placements as critical failures and purges them, ensuring user-error cannot force a collision.

- **Kinematic Singularities (Gimbal Lock):** In the execution layer, trajectories that push the manipulator to the absolute boundary of its dexterous workspace risk kinematic singularities, where the Jacobian matrix determinant approaches zero. Rather than attempting to divide by zero and causing erratic joint behaviour, the OMPL planner within MoveIt 2 successfully identifies the impending singularity. The system handles this mathematically equivalently to a spatial out-of-bounds error, safely triggering a GOAL_STATE_INVALID abortion and transmitting the failure back through the bidirectional OpenIGTLink bridge.

- **Asynchronous Preemption:** In a live clinical setting, a surgeon might rapidly recalculate or "spam-click" new trajectories while the robot is already in motion. Because the ROS 2 listener was engineered using an ActionClient(send_goal_async) rather than a blocking service call, the system inherently supports dynamic preemption. If a new OpenIGTLink matrix arrives mid-movement, the Action Server gracefully cancels the prior goal and mathematically blends a new trajectory from the robot's *current transient state*, completely preventing the violent mechanical jerking associated with legacy open-loop queueing.

4.2 Key Weaknesses and Clinical Limitations

Despite the high mathematical accuracy, the validation process highlighted two critical weaknesses in the current implementation:

1. **Coordinate Frame Ambiguity and Manual Registration:** As observed in the validation of the anatomical visualisation (Figure 6), while the robot correctly reached the mathematical target, the raw STL meshes required a manual quaternion rotation and positional "nudge" within the ROS 2 script to align with the robot's kinematics. This indicates a weakness in the **automated registration** of anatomical meshes. In a clinical setting, relying on manual script-based offsets is a failure point that could lead to significant misalignment if the STL origin is not perfectly standardised across different imaging modalities.
2. **Asynchronous Data Transfer Latency:** The system currently operates as an open-loop execution following a single planning event. The validation of the OpenIGTLink bridge showed that while data integrity is maintained, there is no "real-time" compensation for anatomical movement. If the patient's head moves during the robot's approach, the robot continues toward the original, now-invalidated, coordinates. This "Static Registration" is a major robustness weakness for anything beyond preoperative simulation.
3. **Simulation-Induced Microscopic Joint Desynchronisation:** During stress-testing of the integrated pipeline, a recurrent START_STATE_INVALID error was observed within the MoveIt 2 CheckStartStateBounds planning adapter. Diagnostic analysis of the ros2_control terminal logs revealed this to be a simulation artefact stemming from asynchronous processing delays within the virtual machine environment. Specifically, CPU "overruns" resulted in a failure to command zero-velocity at the exact URDF joint limit during high-speed approaches.

   This latency allowed for a **microscopic momentum drift**, calculated at approximately $< 10^{-4}$ radians, which effectively "pierced" the mathematically absolute joint boundaries defined in the robot model. While this drift remained invisible on the RViz graphical interface, it was sufficient to invalidate the

starting state of the next kinematic request. This phenomenon highlights a fundamental discrepancy between the **idealised mathematical planning** in 3D Slicer and the **stochastic nature of real-time robotic simulation**. In a clinical deployment, the reliance on high-frequency absolute encoders and mechanical hard-stops would preclude such numerical drift, ensuring the manipulator's state remains consistently within the permissible kinematic bounds.

4.3 Future Improvements

To advance this prototype toward a clinical-grade system, several architectural improvements are proposed:

- **Dynamic Registration via Optical Tracking:** Integrating an external tracking system would allow the system to update the world to base_link transform in real-time. This would address the static registration weakness by allowing the robot to "follow" the target if the patient moves.
- **Automated Point-Cloud Registration:** To solve the mesh alignment issue, an **Iterative Closest Point (ICP)** algorithm should be implemented. Instead of manually rotating STLs in Fusion 360 or Python, the system should automatically register the MRI-derived point cloud to the robot's physical sensing of the surgical space.
- **GPU-Accelerated Path Planning:** Moving the distance-map calculations to the GPU would allow for real-time path re-planning. This would enable the system to detect and avoid "new" critical structures (such as moving surgical tools or shifted vasculature) during the mechanical approach.

4.4 Final Conclusion

This project confirms that the integration of 3D Slicer and ROS 2 provides a highly robust, clinically viable framework for image-guided robotics. The system successfully fulfilled all technical requirements: computing an optimal, collision-free trajectory through complex patient anatomy; transmitting that spatial data via a mathematically verified OpenIGTLink bridge; and executing the 6-DOF approach vector with a robotic manipulator.

Furthermore, the implementation went beyond standard open-loop execution by establishing a bidirectional safety architecture. By enabling the robotic execution layer to serialise kinematic failures—such as unreachable workspaces or out-of-bounds start states—and push dynamic GUI alerts back to the preoperative Slicer planner, the system effectively mimics the closed-loop safety standards required of modern medical devices. While the mathematical foundation is sound, future iterations must focus on automating the point-cloud registration process and introducing dynamic optical tracking to ensure total systemic robustness in the face of the unpredictable variables inherent to a live operating theatre.